

Universal Credit Network

A distributed correspondent banking system for remittance routing and such.
v0.0.4

Sections:

0. Introduction

1. System Overview

- 1.1. Monetizing Credit Relationships
- 1.2. Classes of Equity and Maturities of Liabilities
- 1.3. Ledger Management/Balance Sheet Operations
- 1.4. Script System
- 1.5. Routing and Clearing
- 1.6 Transaction Roll-ups
- 1.7. Fees
- 1.8. Fraud Prevention and Detection
- 1.9. Atomic Swaps
- 1.10. Identity Migration
- 1.11. Contract Upgrades
- 1.12. Distributed Hash Table (DHT)

2. Cryptography

- 2.1. Sha256, Sha512, and Shake256
- 2.2. Merkle Tree and Merkle Mountain Range
- 2.3. Directed Acyclic Graph
- 2.4. Ed25519 Digital Signature Scheme
- 2.5. MuSig2 Digital Signature Scheme
- 2.6. Anonymous Multi-Hop Locks
- 2.7. Signature Adapters
- 2.8. AES and XOF Cipher
- 2.9. Curve25519 Elliptic Curve Diffie Hellman Exchange
- 2.10. Ephemeral ECDHE
- 2.11. HashCash with TapeHash

3. Planned Implementation

- 3.1. Contract and Code
- 3.2. Data Models
- 3.3. Suggested Yield Curve
- 3.4. Protocols and Policies
- 3.5. Cryptography
- 3.6. Optimizations

4. Security Considerations

5. Development Roadmap

6. References

7. Glossary

8. Additional Examples

9. License

10. Final Notes

0. Introduction

In this document, I outline an abstract credit network that uses double-entry bookkeeping to maintain ledgers and a specification for a reference implementation in Python. This system does not make use of any global consensus mechanism, but it does use digital signature chains to prove validity, automate clearing of transactions, and to prevent/discover fraud. Correspondents in the network route remittances by clearing (i.e. validating and recording) cryptographically linked transactions consisting of ledger entries and digital signatures. Every pair of correspondents uses 2-of-2 digital signatures to update the state of their credit relation through transactions and to reclaim disk space by periodically combining cleared transactions into transaction roll-ups.

1. System Overview

The system works through identities that extend credit to each other and agree to route payments for each other through the use of Nostro and Vostro accounts tracked in their ledgers. The account representing how much correspondent A owes to correspondent B is tracked as a Nostro asset account in correspondent B's ledger ("our account with you") and as a Vostro liability account in correspondent A's ledger ("your account with us"). Nostro accounts can be thought of as receivable accounts while Vostro accounts can be thought of as payable accounts. Every transaction takes the form of two or more entries into each of two ledgers for which the accounting equation, $Assets = Liabilities + Equity$, is balanced. For correspondent A to remit to correspondent B, correspondent A credits their Nostro account or the Vostro account for correspondent B, offsetting this credit with an equal debit to Equity; correspondent B recognizes this remittance by debiting their Nostro account or the Vostro account for correspondent A and equally crediting their Equity.

A participant can route remittances for their correspondents by clearing pairs of transactions where the inbound value meets or exceeds the outbound value. There are three primary mechanisms by which this can occur: by batch transactions, by hash locks, and through anonymous multi-hop locks (AMHL). Batch transaction involves constructing a batch of entries affecting all ledgers in the payment route, with the only Equity account entries affecting the remitter and the remittee. Hash locks and AMHLs, on the other hand, consist of a chain of transactions each comprised of 4 entries across 2 ledgers. For each transaction in a hash lock remittance route, a 32-byte preimage is required that matches the hash in addition to the relevant signatures. Once the recipient of a hash lock remittance route sees the locked, pending, incoming transaction, he releases the preimage to validate the transaction, and that preimage

then cascades back toward the payment originator to unlock and validate all transactions. For each transaction in an AMHL remittance route, the signature that would make the transaction valid is locked using the AMHL mechanism (a "signature adapter") to guarantee atomicity. Once the setup for the AMHL and all pending transactions in the route is complete, the remitter unlocks the final transaction in the route, and the unlocking of this transaction reveals enough information for the penultimate correspondent to unlock the penultimate transaction. This cascades backward to the originating transaction, unlocking each link with each transaction that clears.

Batch transacting guarantees atomicity in that all hops of the route clear simultaneously or not at all. However, it has the unfortunate property that all hops in a remittance route are linked, eviscerating privacy, and the size of the transaction that each correspondent must retain increases linearly with the number of hops in the route, preventing effective scaling.

Hash locks and anonymous multi-hop locks, on the other hand, require that each intermediary retain just 2 transactions and 8 ledger entries, no matter the length of the remittance route. The hash lock mechanism still theoretically allows for all transactions in a remittance route to be linked, but this would require surveiling all ledgers rather than finding the whole route in a single ledger. The AMHL preserves privacy the best by ensuring that all transactions in the remittance route cannot be linked even by an external observer with access to all ledgers. The hash lock and AMHL were designed with an underlying blockchain providing security for the transaction chain, i.e. a fully-collateralized system with an automatic, decentralized enforcement mechanism. The hash lock/AMHL assumption is that the transaction final clearing/settlement mechanism requires a broadcast of the signed transaction, revealing the value needed for earlier hops to unlock their inbound transactions, but this is not the case in confirming correspondent transactions since there is no blockchain to settle transactions. Some other mechanism is needed to ensure that all hops in a transaction chain are confirmed.

In the proposed system, a transaction that was part of a remittance route is not considered fully cleared until an attestation is received from the sending correspondent(s) or the transaction is included in a periodic transaction roll-up. In this state, it is the obligation of the receiving correspondent to notify the sending correspondent(s) of the confirming of the transaction until an attestation is received from the sending correspondent(s). However, a partial batch transaction may be used to route through correspondents unable or unwilling to take the risk that a network issue delays clearing of the inbound transaction after the outbound transaction clears, and a partial batch transaction may be part of an AMHL route. Additionally, each transaction may include a revocation hash lock provided by the remittance recipient; if the construction of the pending transaction chain fails, the recipient will provide the revocation preimage before the remittance originator attempts remitting through an alternate route. The recipient can also refund a remittance using the associated revocation key until the transaction is included in a transaction roll-up, and correspondent policy will be to prevent propagation of a revocation key within 1 hour of a scheduled transaction roll-up. (Note that the use of a revocation hash lock negates the privacy preservation features of an AMHL.)

Transaction roll-ups are used to condense transactions older than 1 month. These transaction roll-ups are similar to account statements from the legacy financial system and are effectively blocks in a private blockchain for the parties of a correspondence, where each transaction roll-up references the correspondence ID, the previous roll-up ID (except the first, which references the hash of the contract signature that began the correspondence), the root of a Merkle tree containing the IDs of all cleared, non-revoked transactions for the period, any relevant reconciliations, and the aggregate credits and debits to accounts affected by those transactions combined with the aggregates from the previous transaction roll-up. The full list of transaction IDs is maintained as private metadata for the roll-up to create Merkle tree inclusion proofs to prevent attempted double-spending. Validation of a transaction roll-up requires either a multi-party signature from the parties of the correspondence or individual signatures from each. Transactions that lacked attestations are considered fully cleared once included in a transaction roll-up. Finally, transaction roll-ups can be used in correspondent balance sheet audits by producing aggregates that commit to but hide the individual transactions, allowing for all of a participant's correspondents to calculate the equity balance and aggregate credit utilization of the participant. Each participant will maintain a copy of the most recent transaction roll-up for each of that participant's correspondents for redundancy.

As an additional utility, participants in the network will be able to write drafts for offline payments. These drafts are created by a transaction that debits the Equity account and credits a special draft liability account. Once online connectivity is restored, the draft will clear using the normal correspondent clearing mechanism by using an atomic swap.

For compatibility with Bitcoin and similar cryptocurrency systems, a Bitcoin-like script system (bytecode virtual machine) will be used for the transaction validation via locking and unlocking scripts. Each account and draft will be identified by the hash of the locking script(s), and providing a valid unlocking script will enable spending the funds or endorsing/assigning the draft. For accounts, there will be separate locking scripts for debit and credit entries, and the account ID will be the hash of the serialized locking scripts.

Routing through a network of correspondents will be accomplished using a greedy routing algorithm, which uses only local information in deciding how to forward a message. This involves the construction of a spanning tree using a fair root node election process to embed the graph of connections. Intermediate nodes can charge a liquidity use fee for routing remittances, and this fee will be calculated based upon changes in capacity to remit across the correspondence.

The system takes the form of a contract and code designed to execute and enforce the terms of the contract for each correspondent.

1.1. Monetizing Credit Relationships

The premise behind this monetary system is that the economy is the sum of human actions, and a fair system provides equal opportunity and enough flexibility for differing outcomes. This system assumes an equal starting Equity, which is monetized through the extension of credit between correspondents.

Each correspondent relationship will have maximum account limits, each no greater than a set fraction of the Starting Capital asset ("global account limit").

1.1.1. Monetizing Natural Person Correspondents

While all persons can create identities and enter correspondent relationships, only natural persons (humans) are debited with a Starting Capital asset. This Starting Capital asset debit will be offset by an equal credit to the Equity account. The purpose of this restriction is to prevent the creation of potentially fraudulent institutions to inflate the credit supply and profit from seigniorage through graft.

1.1.2. Extending Credit to Correspondents

The creation of a correspondence constitutes the extension of credit to one or both of the counterparties; an agreement to route remittances according to certain rules; and an agreement to ledger constraints and ledger operation constraints. When correspondent A remits to correspondent B, it takes the form of ledger entries indicating that correspondent A owes correspondent B a certain amount of correspondent A's Equity account; thus, to accept a remittance, correspondent B extends credit to correspondent A "collateralized" in a sense by correspondent A's Equity.

1.1.3. Bidirectional and Unidirectional Credit Correspondences

It is possible for a correspondence to be opened in which one party extends credit to the other. This is a way to create custodial accounts where the entity to whom credit is extended holds account balances on behalf of the counterparty ("deposits"). In traditional banking, depositors extend credit to the banking institution they choose to hold their deposits, and the banking institution extends credit to borrowers to whom they choose to lend. In this system, this is all determined by the correspondences.

It is also possible for a correspondence that does not extend any credit to either counterparty for the native currency unit contract, which will be useful for making loans with stated maturities and discount rates, e.g. for vendor financing.

1.1.4. Required Reserves

Correspondents will maintain an Equity balance equal to the global credit limit to prevent insolvency in the case of counterparty failure. Correspondents will not sign transactions that would violate this provision, but they may create adapters for pending outbound transactions that do so linked to pending inbound transactions of equal or greater value.

1.2. Classes of Equity and Maturities of Liabilities

The system will include several classes of equity and maturities for liabilities for flexible funding. In particular, classes of equity will be useful for funding non-natural persons (e.g. credit unions, trusts, private companies, etc). Each correspondent will have two ledgers: one for current accounts and one for future accounts. The standard convention is to denote account types from the future accounts ledger as "Future Assets", "Future Liabilities", and "Future Equity" as appropriate; otherwise, the referenced accounts are from the current accounts ledger.

1.2.1. Classes of Equity

For natural persons, the Equity account will be initially equal to the Starting Capital asset and represents the monetization of the person's labor/economic utility. This is the only class of equity for natural persons, and it is used to fund transactions and receive remittances.

Non-natural persons will be able to issue preferred and common stock to raise capital, and these entries will be placed on the future ledger. Natural persons will be able to exchange their credit in the network for these stocks, increasing the Equity account of the non-natural person to capitalize that ledger. Preferred stock will include a par value, may be callable after a future date, and will entitle the holders to receive dividends (either cumulative or non-cumulative) from the cash flows of the non-natural person when the Equity account exceeds the par value of outstanding common stock + the total dividend amount to disburse. Preferred stock can be called after the temporal constraint has passed and when the Equity account of the non-natural person exceeds the par value of the preferred stock to call + the par value of outstanding common stock + any unpaid cumulative dividends calculated on a pro-rated basis. Common stock will include a par value, may be redeemable under certain conditions, and will entitle the holders to receive dividends from the cash flows of the non-natural person after the preferred stock dividends are paid and an additional Equity buffer is reserved. Dividend payments to common stock holders may be done on a payment-in-kind basis. Dividend payments for both classes of equity will be calculated on a monthly, quarterly, semi-annual, or annual schedule. These classes of equity will be implemented with a secondary contract compatible with the base contract.

1.2.2. Maturities of Liabilities

The base layer monetary instrument/fiduciary medium will be ledger entries representing a perpetual, callable, payment-in-kind, zero-coupon bond with a 0% discount rate, and this will also be the unit of account for the system. All other instruments and maturities will use the base layer monetary instrument for settlement. The proposed implementation will include zero-coupon bonds and notes with positive discount rates to suggest a standard yield curve; the use of discount rates instead of interest rates allows the instruments to be used by those subject to laws prohibiting usury (e.g. the prohibition of *riba* in Islam, which is not transgressed by vendor financing using a discount rate rather than an interest rate).

Since debts are claims on future equity/cash flows, they will not immediately affect the standard Equity account. Instead, a Future Equity account will be used, and it will carry a positive balance for a net creditor and a negative balance for a net debtor. Likewise, the debt instrument ledger entries will use Future Asset and Future Liability accounts. These assets/liabilities will be implemented with secondary contracts compatible with the base contract.

An additional option will be the use of drafts. These drafts will consist of a transaction including entries that debit the Equity account and credit a special draft account, as well as a locking script that requires a signature from the payee. The locking script may include a time lock to create a time draft, lacking which it will be a sight draft. These drafts can be endorsed/assigned to a new locking script or redeemed using atomic swaps and the correspondent clearing mechanism.

1.3. Ledger Management/Balance Sheet Operations

Every transaction in a ledger will take the form of at least 2 ledger entries, and the number of credits will equal the number of debits. The primary ledger operations will be remittances and rebalances. The secondary ledger operations will be amortization to write down accounts of or with inactive correspondents. Below are some examples of how ledgers are affected by transactions, rounding up each value to the nearest integer for simplicity.

Example 1: creating 2 natural person identities + remitting to right correspondent

Assets	=	Liabilities	+	Equity	Assets	=	Liabilities	+	Equity
<i>Starting Capital</i> Dr 10,000				Cr 10,000	<i>Starting Capital</i> Dr 10,000				Cr 10,000
		<i>Vostro</i> Cr 100		Dr 100	<i>Nostro</i> Dr 100				Cr 100
<i>10,000</i>		<i>100</i>		<i>9,900</i>	<i>10,100</i>				<i>10,100</i>

Example 2: routing a remittance through correspondent B

Assets	=	Liabilities	+	Equity
<i>Starting Capital</i> Dr 10,000				Cr 10,000
<i>Nostro A</i> Dr 101				Cr 101
		<i>Vostro C</i>		Dr 100

	Cr 100	
10,101	100	10,001

Example 3: correspondent B rebalancing to restore liquidity with A and C

Assets	=	Liabilities	+	Equity
<i>Starting Capital</i> Dr 10,000				Cr 10,000
<i>Nostro A</i> Dr 101		<i>Vostro C</i> Cr 100		Cr 1
<i>Nostro C</i> Dr 507		<i>Vostro A</i> Cr 500		Cr 7
<i>Nostro A</i> Cr 101		<i>Vostro A</i> Dr 101		
<i>Nostro C</i> Cr 100		<i>Vostro C</i> Dr 100		
<i>Nostro A</i> 0		<i>Vostro A</i> 399		
<i>Nostro C</i> 407		<i>Vostro C</i> 0		
10,407		399		10,008

Example 4: issuing and receiving zero-coupon bonds (perspective of correspondent B)

Assets	=	Liabilities	+	Equity	Future Assets	=	Future Liabilities	+	Future Equity
<i>Starting Capital</i> Dr 10,000				Cr 10,000					
<i>Nostro A</i> Dr 1,000				Cr 1,000			<i>Q-Bond A</i> Cr 1,013		Dr 1013
		<i>Vostro C</i> Cr 1,000		Dr 1,000	<i>H-Bond C</i> Dr 1,028				Cr 1028
							<i>Q-Bond A</i> Dr 1,013 Cr 1,026		Dr 13
11,000		1,000		10,000	1,028		1,026		2

Issuing of a quarter bond (*Q-Bond*) to fund the writing of a half-year bond (*H-Bond*) involves rolling over the *Q-Bond*, incurring an additional discount (an additional 13 for a total paid discount of 26) and incurring funding risk. The margin for the operation is just 20 basis points. Whether or not positive maturity transformation is economical at these rates will depend on funding and counterparty risks, and of course discount rates may vary from the reference model yield curve.

Example 5: amortizing to write down accounts of/with inactive correspondents

Assets	= Liabilities	+ Equity
<i>Starting Capital</i> Dr 10,000		Cr 10,000
<i>Nostro A</i> Dr 12	<i>Vostro C</i> Cr 24	Dr 12
<i>Nostro A</i> Cr 1	<i>Vostro C</i> Dr 2	Cr 1
<i>Nostro A</i> Cr 1	<i>Vostro C</i> Dr 2	Cr 1
<i>Nostro A</i> 10	<i>Vostro C</i> 20	
10,010	20	9,990

Amortization to write down accounts of/with inactive correspondents will be limited to cases where the last transaction on the ledger regarding a correspondence is at least one year old and attestations of the same from three other of that counterparty's correspondents are acquired or when the end timestamp of the correspondence has been passed without renewal. It thus takes four correspondents to mark a mutual counterparty as inactive to initiate the amortization protocol. The amortization process will mark down these accounts evenly over the course of a year. In the case of a correspondent failure due to a technological issue and loss of private key seed, identity migration will structure the recreation of the old ledger using zero-coupon notes to smooth out the recovery process by matching maturities for some amount of the account values with the write-down transactions.

Example 6: issuing note and amortizing to write-down a migrated identity

Assets	= Liabilities	+ Equity	Future Assets	= Future Liabilities	+ Future Equity
<i>Starting Capital</i> Dr 10,000		Cr 10,000			
<i>Nostro Old</i> Dr 100	<i>Vostro Old</i> Cr 1,000	Dr 900			

<i>Nostro New</i> Dr 100	<i>Vostro New</i> Cr 100		<i>Y-Note New*</i> <i>Cr 900</i>	Dr 900
<i>Nostro Old</i> Cr 8	<i>Vostro Old</i> Dr 83	Cr 75		
	<i>Vostro New</i> Cr 75	Dr 75	<i>Y-Note New</i> Dr 75	Cr 75
<i>Nostro Old</i> Cr 8	<i>Vostro Old</i> Dr 83 <i>Vostro New</i> Cr 75		<i>Y-Note New</i> Dr 75	Cr 75
<i>[10 period sum]</i>				
<i>Nostro Old</i> Cr 80	<i>Vostro Old</i> Dr 830 <i>Vostro New</i> Cr 750		<i>Y-Note New</i> Dr 750	Cr 750
<i>Nostro Old</i> Cr 4	<i>Vostro Old</i> Dr 4			
<i>10,100</i>	<i>1,000</i>	<i>9,100</i>	<i>0</i>	<i>0</i>

* The Y-note issuance transaction will be post-dated to make the payments coincide with the amortization schedule. Alternately, a variable-length zero coupon bond can be issued that converts into a Y-note upon maturation, both of which are valid only if the old identity remains inactive. No fees or discounts are applied to this operation. It takes two years to restore the liquidity and credit as it existed at the time the original identity private key was lost.

Example 7: routing remittances to borrow and pay down a note (perspective of correspondent A)

Assets	=	Liabilities	+	Equity	Future Assets	=	Future Liabilities	+	Future Equity
<i>Starting Capital</i> Dr 10,000				Cr 10,000					
<i>Nostro B</i> Dr 1,000				Cr 1,000			<i>W-Note X</i> Cr 1,001		Dr 1,001
<i>Nostro B</i> Cr 143				Dr 143			<i>W-Note X</i> Dr 143		Cr 143
<i>Nostro C</i> Cr 143				Dr 143			<i>W-Note X</i> Dr 143		Cr 143
<i>Nostro B</i> Cr 143				Dr 143			<i>W-Note X</i> Dr 143		Cr 143
		<i>Vostro D</i>		Dr 143			<i>W-Note X</i>		Cr 143

	Cr 143			Dr 143	
<i>Nostro B</i>		Dr 143		<i>W-Note X</i>	Cr 143
Cr 143				Dr 143	
	<i>Vostro D</i>	Dr 143		<i>W-Note X</i>	Cr 143
	Cr 143			Dr 143	
<i>Nostro C</i>		Dr 143		<i>W-Note X</i>	Cr 143
Cr 143				Dr 143	
10,285	286	9,999	0	0	0

Using AMHLs, a transaction debiting the *W-Note X* account on Ledger A and crediting the *W-Note A* account in Ledger X could be locked with the output of a remittance route -- the transaction affecting the *W-Note* accounts would form the first link in the remittance route.

Example 8: creating, endorsing (splitting), and redeeming a draft

Assets	= Liabilities	+ Equity	Issued Drafts
<i>Starting Capital</i>		Cr 10,000	
Dr 10,000			
	<i>Draft Account 1af3</i>	Dr 125	<i>1af3</i>
	Cr 125		+125
			<i>1af3</i>
			-50
			<i>1af3->b43c</i>
			+50
	<i>Draft Account 1af3</i>		<i>1af3->b43c</i>
	Dr 50		-50
	<i>Vostro A</i>	Dr 1	
	Cr 51		
	<i>Draft Account 1af3</i>		<i>1af3</i>
	Dr 75		-75
	<i>Vostro B</i>	Dr 1	
	Cr 76		
10,000	127	9,873	0

In this example, the original draft is partially endorsed (the value split) prior to redemption, and the costs of clearing the payments to redeem the drafts are paid by the draft writer. The endorsement of drafts and splitting of their values takes place using a Bitcoin-style transaction model and does not affect the balances of ledger accounts. This endorsement feature may be useful for adapting to temporary liquidity constraints in the clearing network, for example by swapping the draft (perhaps at a discount) with someone who expects to have adequate liquidity to redeem the draft in the near future.

1.4. Script System

When Satoshi Nakamoto wrote the Bitcoin white paper, he did not include any specification for a script system to enable programmatic encumbrances, but he included a basic, Forth-like, stack-based, Turing-incomplete script system in the implementation. History has shown that the script system was a worthy addition, though it has required several updates to enable broader use. This system will use a similar script system for creating encumbrances on accounts and drafts.

Encumbrances take the form of a locking script applied on an account or on a draft/UTXO. The script system will be similar to that used in Bitcoin: execution occurs upon values held in a stack implemented as a last-in-first-out queue; the unlocking script is executed before the locking script; if the queue has a single value that evaluates to not-false after both scripts have executed, the encumbrance has been fulfilled. All values and op codes are encoded as a stream of bytes and interpreted by the script interpreter. The script will be designed to be conceptually compatible with the Bitcoin script system, including some currently disabled op codes and pending proposals, so that useful Bitcoin layer 2 technologies can be integrated if needed.

A script transpiler will be included which will convert human-readable code into byte code and vice versa. Anything between hashtags or quotation marks will be ignored by the transpiler as comments. Values starting with a “d” will be interpreted as a decimal value, while values starting with an “x” will be interpreted as a hexadecimal value. See the tapescript project on PyPI for a working library of this system: <https://pypi.org/project/tapescript>.

1.4.1. Account Encumbrances

Nostro and Vostro accounts require a valid MuSig multi-party signature or a signature from each correspondent to create a valid transaction. The locking and unlocking scripts thus take the following form.

```
# committed script #
IF {
  PUSH x<hex musig pubkey>
  CHECK_SIG
} ELSE {
  PUSH x<hex pubkey1>
  PUSH x<hex pubkey2>
  SWAP 1 2
  CHECK_SIG_VERIFY
  CHECK_SIG
}

# locking script #
DUP
SHAKE256 d20
```

```
PUSH x<hex committed script shake256 hash>
EQUAL_VERIFY
EVAL
```

```
# unlocking script: musig #
PUSH x<hex signature from musig pubkey>
TRUE
PUSH x<hex committed script src>
```

```
# unlocking script: 2 signatures #
PUSH x<hex signature from pubkey1>
PUSH x<hex signature from pubkey2>
FALSE
PUSH x<hex committed script src>
```

Other accounts may be setup with arbitrary locking scripts. Each account will include a signature from the relevant ledger owner(s) authorizing the account. The tapescript library linked above includes 90 ops, a soft-fork system for extending with new ops, dozens of tools, 104 test vectors for compilation and decompilation of various locking scripts, and end-to-end tests of dozens of different script types.

1.4.2. Draft Encumbrances

There are two types of encumbrances related to drafts: encumbrances on draft accounts that hold the funds for specific drafts, and encumbrances on the drafts themselves. The former takes the form of a locking script that uses OP_CHECK_TRANSFER, and the latter takes the form of a pay-to-pubkey (P2PK), pay-to-pubkey-hash (P2PKH), pay-to-script-hash (P2SH), hash time lock (HTLC), or point time lock (PTLC). (The implementations of these locking script types will differ from the Bitcoin equivalents.)

1.4.3. Contract Interoperability

Contract interoperability will be supported by the implementation of an OP_CHECK_TRANSFER opcode in the script interpreter of each contract. This opcode takes a byte argument as an unsigned integer *count*; pulls the top item from the stack as a *contract_hash*; pulls the next item from the stack as an unsigned integer *amount*; pulls the next item from the stack as a serialized transaction *constraint* (e.g. a funding source); pulls the next item from the stack as a serialized *destination*; pulls *count* number of items from the stack as *sources*; pulls *count* number of items from the stack as serialized *transfer_proofs*; then uses the contract specified to verify that each transfer proof is valid and that the aggregate sent to the destination is greater than or equal to the *amount*. This operation produces an error if the specified contract was not loaded into the execution environment, or if the specified contract does not fulfill the interface (i.e. it must include certain functions).

Example: credit default swap:

```
# committed script: eltoo-like multisig contract #
OP_IF {
```

```

OP_PUSH d<unix epoch of maturation + grace period>
OP_DUP
OP_CHECK_TIMESTAMP_VERIFY
OP_CHECK_EPOCH_VERIFY
OP_PUSH x<funding source>
OP_PUSH x<funding destination>
OP_PUSH x<encoded txn constraints>
OP_PUSH d<amount>
OP_PUSH x<hex contract hash>
OP_CHECK_TRANSFER 1
OP_VERIFY
OP_PUSH x<hex pubkey_CDS_purchaser>
OP_CHECK_SIG
} ELSE {
  OP_IF {
    OP_PUSH d<unix epoch of maturation + grace period + 30 days>
    OP_CHECK_EPOCH_VERIFY
    OP_PUSH x<hex pubkey_CDS_issuer>
    OP_CHECK_SIG
  } ELSE {
    OP_PUSH x<hex pubkey_2of2>
    OP_CHECK_SIG
  }
}

```

unlocking script: CDS redemption

```

OP_PUSH x<hex signature from pubkey_CDS_purchaser>
OP_PUSH x<hex transfer proof from contract>
OP_TRUE
OP_PUSH x<hex committed P2PK script src>

```

unlocking script: CDS expiration

```

OP_PUSH x<hex signature from pubkey_CDS_issuer>
OP_TRUE
OP_FALSE
OP_PUSH x<hex committed P2PK script src>

```

unlocking script: CDS transfer

```

OP_PUSH x<hex signature from pubkey_2of2>
OP_FALSE
OP_FALSE
OP_PUSH x<hex committed P2PK script src>

```

locking script

```

OP_DUP
OP_SHAKE256 20
OP_PUSH x<hex committed script shake256 hash>
OP_EQUAL_VERIFY
OP_EVAL

```

1.5. Routing and Clearing

1.5.1. Definitions

Inbound Transaction - A transaction that credits the Equity account with respect to a correspondent.

Outbound Transaction - A transaction that debits the Equity account with respect to a correspondent.

Link - A line of credit between correspondents used for clearing transactions.

Lock - A value that is used as a curve point to create an adapter signature, or the action of creating an adapter signature for a given transaction using a lock value.

Release - A value that is used as a scalar to decrypt an adapter signature, or the action of decrypting an adapter signature for a given transaction using a release value.

Revocation Key - A 32-byte random value generated by a recipient to revoke pending transactions from a failed remittance route operation.

Revocation Hash Lock - The 32-byte sha256 digest of the Revocation Key.

Pending Transaction - A transaction that has been locked.

Cleared Transaction - A transaction that has been released.

Transaction Roll-up - A signed data block committing to a transaction history while revealing only the aggregate account balances as they were calculated at the specified time for the roll-up period.

1.5.2. Route Locking

The route locking convention is that correspondents will sign inbound transactions without precondition, but correspondents will create adapter signatures for outbound transactions in a route upon witnessing a pending inbound transaction that uses a connected lock. See 2.4 and 2.5 for details on the cryptography underlying this mechanism.

1.5.3. Route Discovery

Routing will use an adaptation of the SpeedyMurmurs Path-Based Transaction (PBT) network routing scheme, built upon the Virtual Overlays Using Tree Embeddings (VOUTE) system.

The core concept is that a connected graph can be organized using spanning trees to derive coordinates (an embedding): once a root is selected for a spanning tree, that node gets an empty coordinate as its address, and that root's neighbors (directly connected peers; i.e. 1-hop peers) each pick a random b -bit number and request that coordinate assignment from the root; each node then advertises its new address to its neighbors, and those neighbors do the same, appending their b -bit numbers to the coordinate of the parent node and requesting assignment of the resulting address; "greedy" routing is then accomplished by traversing the tree and using shortcuts, forwarding packets to the neighbors with the shortest distance or greatest common address prefix length to the destination. For this system, coordinates will be encoded as address components using big endian bit order, and each node will append components of null bits to the end of its assigned address until it reaches the proper length.

Routing can be accomplished with one of two distance metrics: tree distance (TD) and common prefix length distance. Tree distance is defined as $\delta_{TD}(x_1, x_2) = |x_1| + |x_2| - 2cpl(x_1, x_2)$ where cpl denotes common prefix length measured in address components and $|x_1|$ denotes the number of non-null address components. Common prefix length (CPL) distance is defined as $\delta_{CPL}(x_1, x_2) = L - cpl(x_1, x_2) - 1 / (|x_1| + |x_2| + 1)$ for $x_1 \neq x_2$ and 0 if $x_1 = x_2$. Routers then forward messages by choosing the neighbor with the smallest distance calculated with the metric specified in the packet destination information.

Stefanie Roos et al. also devised a system of anonymous, route-preserving return addresses that can be used as an adaptation of VOUTE. They proposed using addresses with 128-bit components and up to $L=128$ components, leading to addresses in each embedding of up to 2048 bytes, which is a substantial use of router resources (bandwidth and memory), especially when using multiple embeddings. A route preserving anonymous address is created by padding the address with pseudorandom components (overwriting the trailing null bits) and applying a cascading hash function: first, the node chooses a 256-bit random seed s_{pad} ; second, the s_{pad} is passed into a pseudorandom number generator (PRNG) XORed with the index of the address component to generate ($a'_i = PRNG(s_{pad} \oplus i)$ for i where $a_i = 0$); third, the generated address components are used to overwrite the trailing null bits of the address, resulting in padded address $a' = a'_0 || \dots || a'_L$; fourth, the node chooses a 256-bit random value k ; finally, a cascading hash hc is applied: $hc(a', k) = d_0 || d_1 || \dots || d_L$ where $d_0 = h(k \oplus a'_0)$ and $d_j = h(d_{j-1} \oplus a'_j)$ for $0 < j < L$. ($\oplus$ is the XOR operation, $||$ denotes concatenation, and L is the number of components in an address.) The resulting address is then (k, y) where $y = hc(a', k)$.

Modified distance metrics are defined for using route-preserving, anonymous addresses: $\delta_{RP-*}(k, y, c) = \delta_*(y, hc(c, k))$; i.e. apply the cascading hash function hc to address c of a neighbor and calculate distance using the original distance metric. This can be optimized by applying the cascading hash only up to the first address component of $hc(c, k)$ where $hc(c, k)_i \neq y_i$.

This system will use 4 spanning tree embeddings with addresses comprised of 8-bit components where $L=32$ and 4 spanning tree embeddings with addresses comprised of 16-bit components where $L=16$ (256-bit addresses). In addition, the highest value for each component is reserved for an extended child address set, allowing each parent in the tree embedding to allocate an additional number of child addresses approximately equal to $(L \text{ less the number of previously used components}) * 255$. Route-

preserving, anonymous addresses will not be employed in the basic implementation due to resource constraints and the additional complexity involved, but it may be implemented as a future upgrade.

Root selection will employ proof-of-luck system: each of the 8 spanning trees will have a root target byte string; when a node starts up, it calculates its distance from each root target by XORing its public key bytes with the target byte string and interpreting the result as an integer; when the node connects, it requests the currently elected roots for each spanning tree, calculates the distances, and compares against itself; if it has a smaller distance from any target byte string, it claims root status for that spanning tree for itself; each node accepts the claim with the smallest distance from the root target, requesting a new address for the new spanning tree from its (possibly new) parent in the tree, and promulgating the root election, its new address, and the chain of digital signatures to its neighbors. The use of 8 embeddings and CPL routing should prevent more than 5% route failure rate even if an attacker corrupts the root selection process by recomputing its key pair in a manner similar to proof-of-work.

When a node wishes to send a message to another node, it selects the distance metric to use for routing to the destination (0 for δ_{TD} or 1 for δ_{CPL}); it then selects between 1 and 8 embeddings to route through for delivery and between 1 and 8 embeddings in which it will accept a response, identifying each by setting a bit with index equal to its index in the sorted list of root targets (i.e. the first embedding is associated with the first bit, the second embedding with the second bit, etc). An encoded address will start with a byte encoding the selected embedding (i.e. with a single bit set) followed by the 32 bytes of the embedding address. As an optimization, the first byte of each encoded address can be combined via bitwise OR, followed by the addresses concatenated in the same order as used in the embedding identification step. The packet destination will be a byte string comprised of the selected embeddings ORed together into a 1-byte integer form followed by the associated 32-byte address for every embedding chosen. The packet source will be similarly constructed. Thus, the minimum length for a destination or return component will be 33 bytes, and the maximum length will be 257 bytes. Nodes will route messages through the correspondent network by selecting the correspondent with the lowest distance across all available embeddings of those specified in the destination address component.

To route a remittance using these embeddings, route discovery packets will be sent to the destination address. Each node that encounters the packet will return a message to the originator containing the top N candidates for the next hop in the route, whether or not the correspondent link has enough liquidity to facilitate the remittance and with associated fee rates. Nodes will append their transaction fee to the route discovery packet before forwarding so that other nodes in the route can calculate liquidity properly. If a node cannot facilitate the remittance, it returns an error to the previous hop, which then forwards the route discovery packet to the next best hop; if all possible hops are exhausted, the error propagates backward with the list of correspondent IDs that could not forward the remittance. This forwarding and backtracking continues until a functioning route is found or the error is returned to the remittance originator, indicating that no route could be found for the distance metric and embeddings specified.

Once a suitable remittance route has been found, the originator can then produce the anonymous multi-hop locks, sending the locks to each node in the route. Once each node in the remittance route has verified the lock, constructed a pending transaction using the lock, and responded affirmatively to the originator, the originator releases the lock, and the transactions clear. For privacy, remittances can be split into 2 to 4 randomly-distributed chunks routed over different combinations of embeddings.

1.5.4. Revocation Hash Lock

Before a remittance route is created, the recipient will create a secret revocation key and share its sha256 digest with the remittance originator. This digest will comprise the revocation hash lock. If the revocation key is revealed at any time after the creation of pending transactions within a remittance route but before a transaction roll-up finalizes the transactions, the hash lock will be released and the transactions will be revoked.

1.5.5. Settlement

If a participant exhausts its liquidity in a correspondence, it can rebalance as shown in 1.3, Example 3. If a participant maintains a net debtor balance with a correspondent and has not transacted with a correspondent in six months, the correspondent can call the loan and require payment-in-kind through the network. If a participant wishes to cease transacting on the network, that participant must settle all accounts with correspondents by zeroing out all balances; this may include out-of-band settlement means, such as payment with sovereign currency, Bitcoin, gold, or some other monetary asset. In general, real economic transactions of goods and services are the mechanism of settling balances.

1.5.6. Delegated Routers

To increase the availability of a ledger for routing and accepting incoming transactions and remittances, a participant may delegate authority to handle Equity-neutral/positive remittance routing and countersigning of Equity-positive pending transactions to a dedicated router. A primary router is sponsored and operated by the participant while a secondary router is sponsored and operated by a correspondent. The use of primary routers allows for a participant to engage in remittance routing without keeping the keys necessary for spending loaded in memory, allowing use of hardware "cold" wallets without impacting routing capabilities, while the use of secondary routers provides redundancy and higher availability. When a participant signs a router delegation, they share it with their correspondents and their other routers so that the correspondents know to use the router for routing remittances and the other routers know to synchronize transaction histories (and optionally perform load balancing) with the new router. Secondary routers can sign only incoming transactions and batch transactions to guarantee that each transaction authorized has a neutral or positive effect on the participant's Equity account.

Periodically, each router will synchronize transactions with the participant and any other routers for that participant. By convention, this will occur when the participant or router initially connects to the

network and once every 10 minutes thereafter. Delegated routers will also maintain copies of the most recent transaction roll-up for each correspondent.

Router operators will be compensated from additional transaction fees by including entries collecting those fees during batch transactions and/or by being one of the hops of the route. If operating a secondary router is a profitable endeavor, more participants will be incentivized to operate routers and compete, driving down router fees; if it becomes unprofitable, router operators will be incentivized to cease routing, restoring profitability by reducing competition. It should balance naturally through market forces.

1.6. Transaction Roll-ups

Transaction roll-ups (TxRUs) are summaries of the transaction history of the accounts involved in a correspondence and a finalization of non-revoked transactions. Each TxRU references the correspondence, the previous transaction roll-up (or the hash of the multisig of the contract for the first TxRU), the Merkle root of the txns included in the TxRU, and the net balances of the accounts as calculated by applying the entries of the included txns and the net balances of the previous TxRU to the associated accounts. TxRUs provide a snapshot of the current correspondent accounts as they stood at the end of the roll-up period without revealing any underlying txns and can be used by a participant's correspondents to audit that participant's balance sheet to ensure that the participant is solvent and meets the reserve requirement.

The txn Merkle root is created by using the txn IDs as leaves in a Merkle tree structure. This system will use the Merkle mountain range construction and pass in the txn IDs ordered by ascending ID. The correspondents who own the accounts referenced in the TxRU will attach the ordered list of leaves to their record of the TxRU, but this list will be omitted when the TxRU is shared with other correspondents.

TxRUs will be produced every week, and each will include all transactions confirmed since the previous TxRU and at least 7 days old.

1.7. Fees

Routing of remittances will incur fees to compensate correspondents and incentivize routers. These fees will take two forms: a small fixed fee for the bandwidth, computational, and data storage resources used in routing and clearing the transactions; and a liquidity use fee that scales with the effect on the liquidity of the links used. The fixed fee will incentivize use of shorter routes and less network resources; the liquidity use fee will incentivize routing that maximizes residual bidirectional liquidity in the network.

After clearing of a transaction, the liquidity of a given link can either increase or decrease. The liquidity use fee will be at most 0 for links that experience increased liquidity as a result of a transaction, and it will be calculated according to the following formula for links that experience decreased liquidity:

$$\text{fee_rate} = \int_{1-\text{start liquidity}}^{1-\text{end liquidity}} (x^2) dx, \text{ where } \text{start liquidity} \text{ is the directional link liquidity before the}$$

transaction expressed as a proportion of the credit limit and *end liquidity* is the directional link liquidity after the transaction also expressed as a proportion of the credit limit; i.e. $f(x) = (x^3)/3$ and $\text{fee_rate} = f(1 - \text{end liquidity}) - f(1 - \text{start liquidity})$. For the purposes of this calculation, directional liquidity in excess of the configured credit limit is free to use; i.e. $\text{start liquidity} = \min(1, \text{link liquidity} / \text{link limit})$ and $\text{end liquidity} = \min(1, (\text{link liquidity} - \text{txn amount}) / \text{link limit})$. The maximum fee rate possible will be 50% for when link liquidity drains from 100% to 0%. To incentivize routing through a correspondent that needs its links rebalanced, correspondents may optionally allow for negative transaction fees to be calculated as some proportion of the above formula; the default options for this will be half the above formula and will be no less than -1% (i.e. $-1\% \leq \text{fee_rate} \leq 50\%$).

1.8. Fraud Prevention and Detection

Fraud cannot be entirely prevented in any system. Even with Bitcoin, fraud is still possible in some circumstances (e.g. during a network partition, 51% attack, with certain types of 0-confirmation transactions, or against poorly-implemented SPV nodes). However, this system limits the amount of damage that can be inflicted by a fraudulent actor, and it will be possible to detect and prove fraud in some instances using the digital signature chains.

1.8.1. Types of Fraud

There are several ways to defraud counterparties: repudiation, false representations, and exceeding balance sheet constraints are the primary forms. Secondary forms of fraud are creating additional identities and creating fictitious correspondents. Additional identities will require unique correspondences to exceed the credit monetization limits/ledger constraints, so the potential damage should be limited by the ability of the individual to gain the trust of potential correspondents. Both of the secondary forms of fraud are varieties of the Sybil attack and discussed in later sections.

Repudiation occurs when a correspondent violates the terms of the contract and drops from the network without first settling outstanding account balances.

False representations include providing falsified Know Your Correspondent (KYC) data or falsified attestations to correspondents.

Exceeding balance sheet constraints means making transactions that deplete the Equity account below the reserve requirement.

1.8.2. Fraud Prevention

The enforcement mechanism to prevent repudiation is the implicit threat of real-world consequences for violating the terms of the contract. In a state with rule of law, this takes the form of legal actions performed by courts of competent jurisdiction. In a state of anarchy, this takes the form of reputational damage and future exclusion from the network.

The primary mechanism to prevent false representations is for participants to perform due-diligence to confirm the KYC information provided before signing the correspondent contract. Secondly, the above enforcement mechanisms can be employed.

The primary mechanism for preventing a correspondent from exceeding balance sheet constraints will be the aggregation of transaction roll-ups by the correspondents of the participant to be audited. Each transaction roll-up presents a statement attested by both the target of the audit and the relevant correspondent, and aggregating the effects on the Equity account is a straight-forward calculation. This aggregate calculation will be used to determine whether or not a correspondent can fund a specific transaction, and correspondents will not clear transactions that would decrease the participant's Equity account below the reserve requirement. The enforcement mechanisms outlined in the first paragraph also apply.

1.8.3. Limitations on Damage

The damage that can be inflicted by a fraudulent correspondent is limited to the sum of the credit limits of their correspondences. The reserve requirement prevents a single fraudulent or insolvent actor from causing another correspondent to fail, and the amortization system will smooth out the balance sheet effects over the course of a year in the case where a correspondent drops from the system with unsettled/unrebalanced correspondences.

1.8.4. Fraud Detection

Detecting additional identities will be possible if a person committing such fraud accidentally exposes more than one identity to a given correspondent. The software will not make the management of multiple identities an easy task; for someone to pull this off, they will require multiple devices to host the multiple identities and a careful methodology for preventing any accidental exposures or fumbling with multiple devices to rebalance a correspondence.

Fictitious correspondences can be detected by searching for routes to those participants. If all routes to a participant are through a specific correspondent given a sufficiently large and well connected network, it is a warning sign that the participant may be a fictitious correspondent. The participant can then be messaged over the network to attempt to establish whether or not the participant is a real person, and appropriate legal actions may be taken against a correspondent if necessary to prevent or punish fraudulent activity.

1.9. Atomic Swaps

Atomic swaps are possible with cryptocurrency systems using an adaptation of the hash time locked contracts (HTLCs) and point time locked contracts (PTLCs). These will also be used to clear drafts via the correspondent network.

An HTLC is created by putting value in a UTXO or account with a locking script with a payment clause that allows spending by a public key + a hash preimage and a refund clause that allows spending by the refund public key after a specific time has passed. By creating two HTLCs that share the hash lock and time lock, it is possible to swap two assets trustlessly between trade participants or route payments through a correspondent payment network. See section 2.1 Sha256, Sha512, and Shake256 for more information about the hashing used in hash locks.

A PTLC is created by putting value in a UTXO or account with a locking script with a payment clause that allows spending by a tweaked public key and a refund clause that allows spending by the refund public key after a specific time has passed. See section 2.7 Signature Adapters for more information about key pair tweaking.

1.10. Identity Migration

@todo specify

1.11. Contract Upgrades

Correspondents may upgrade the contract they use for clearing. Such upgrades may be compatible with the prior contract version. To upgrade a correspondence, the correspondents first sign a new correspondence relationship using the new contract, then migrate their account balances from the old contract to the new one using atomic swaps, then revoke the old correspondence.

1.12. Distributed Hash Table (DHT)

A Distributed Hash Table will be used to maintain public records of router delegations as part of public profiles for participants. @todo specify

2. Cryptography

2.1. Sha256, Sha512, and Shake256

Sha256 is the 256-bit output version of the Secure Hash Algorithm 2 (SHA-2). Sha512 is the 512-bit output version of SHA-2. Shake256 is the extensible output function (XOF) derived from the Secure Hash Algorithm 3 (SHA-3) using a 256-bit block size. The purpose of sha256 is to take any input byte string of any length and output a byte string of length 32 (64 for sha512), where the change of any 1 bit in the input changes approximately half of the bits of the output in an unpredictable manner. Shake256 has a similar property of cascading unpredictability in the output due to any slight change in the input, but it has a configurable output length rather than a fixed output length.

SHA-2 and SHA-3 families of functions are believed to be secure one-way functions; there is a lot of statistical evidence to support this, but there is no formal mathematical proof that one-way functions exist or that SHA-2 or SHA-3 perfectly fulfill the requirements. However, many years of practice and cryptanalytical attacks have proven the robustness and security of these functions.

2.2. Merkle Tree and Merkle Mountain Range

A Merkle Tree is a data structure that commits a collection of data to a single hash output and allows for creation of efficient inclusion proofs. It is constructed as follows: given a collection of data, take the hash output of each one denoted leaves; hash together two leaves concatenated together to form an intermediate hash denoted a node; hash together nodes two at a time to form more nodes until only one hash remains, and that is the root. An inclusion proof can then be generated for any individual datum by providing it, the other leaf it is combined with, and a series of nodes that combine at each level to eventually form the root. Security of this system relies upon the security of the underlying hash function; this system will use sha256. Check code reference 1 for an example of how this works in practice.

2.3. Direct Acyclic Graphs

A directed, acyclic graph (DAG) is a graph composed of nodes connected together with directed edges where no cycle of directed edges exists. A node is merely a collection of data, and an edge is merely a reference to a node. Creating a DAG is easily accomplished by using the cryptographic hash of the node contents as the node identifier, then encoding those node identifiers in edges contained in a node's content; since the identifier must be known before a node's contents can be calculated, and that identifier cannot be predicted, and any change in a node's contents changes the identifier, it is impossible for any number of nodes to form a cycle.

DAGs are used for tracking the transfer of ownership of Bitcoin and Bitcoin-derived cryptocurrencies. The structure of a DAG does not itself prevent the creation of nodes or the double-spending of coins, so the methods for adding to a DAG must include such preventative mechanisms. In Bitcoin, the permission to add to the DAG comes in the form of being able to unlock an unspent transaction output (UTXO), and the prevention of double-spending is done by network validation using the Nakamoto

consensus mechanism (proof-of-work) to provide a distributed timestamp/notary service that orders all transactions (only the first valid transaction attempting to spend a UTXO settles).

2.4. Ed25519: Twisted-Edwards Digital Signature Algorithm (EdDSA)

The Ed25519 Digital Signature Algorithm (EdDSA) was created by Daniel J. Bernstein et al. as an efficient method of producing digital signatures using the Schnorr signature construction over the Twisted Edwards elliptic curve where $q = 2^{255}-19$ for defining the Schnorr group of valid points. Elliptic curve points can be combined using the group operator (denoted algebraically as addition) and are created by multiplying a scalar (255-bit integer) with a base point (also called the group generator), which is defined as applying the group operator to the base point the scalar number of times.

The group operator for curve points is defined as an algebraic: given the Edwards curve parameters E, a, d and two points $P_1=(x_1, y_1)$ and $P_2=(x_2, y_2)$, a third point $P_3=(x_3, y_3)$ can be calculated as follows: $x_3 = (x_1y_2 + y_1x_2)/(1 + dx_1x_2y_1y_2)$, $y_3 = (y_1y_2 - ax_1x_2)/(1 - dx_1x_2y_1y_2)$. The authors devised the algorithm to be highly secure and efficient using common hardware. The behavior of the group operation results in a fast, stochastic, unpredictable, and deterministic (thus pseudo-random) output. This stochastic but deterministic behavior is the basis of the one-way quality of the scalar-point multiplication function.

Multiplying a scalar by a curve point is a one-way function: given two points P_1 and P_2 , it is impossible to find a scalar s such that $P_1(s) = P_2$ except with a brute-force search of the scalar field, which contains approximately 2^{255} ($\sim 5.8 \cdot 10^{76}$) members. It also has homomorphic properties: given a curve point P and two scalars s_1 and s_2 , $P(s_1+s_2) = P(s_1) + P(s_2)$; also, $P(s_1*s_2) = P(s_1)(s_2) = P(s_2)(s_1)$. The domain of this function is 255-bit integers with some constraints applied to some of the bits, depending upon which version of the function is used (clamped or unclamped; for key generation or nonce generation); as such, the input space is a finite field modulo L (values greater than L wrap around from 0; L is defined in the EdDSA standard RFC 8032). The co-domain of this function is comprised of a Schnorr group of elliptic curve points encoded as a 255-bit x coordinate and a sign bit for the y coordinate (each x coordinate can be paired with a positive or negative y coordinate of a defined magnitude).

To create a signature, two scalars, a private key x and nonce seed k , are first derived from a 32-byte seed: $x = H(\text{seed})[:32]$; $k = H(\text{seed})[32:]$, where H denotes the SHA512 hash function. The private key is then multiplied by the base point G to create the public key X : $X = G(x)$. Then the seed nonce k and the hash of the message m to be signed are combined and hashed to produce a secret scalar denoted r : $r = H(m || k)$. (This is called the Fiat-Shamir heuristic; a different method is used to create the nonces for multisig.) Then, r is multiplied with the base point to create the public nonce R : $R = G(r)$. A challenge scalar c is then created by hashing the nonce R , the public key X , and the message m : $c = H(R || X || m)$. Finally, the signature is created by calculating a scalar s : $s = cx + r$. The signature for message m is then (R, s) and can be confirmed by recalculating c from public values and checking if the following condition holds: $G(s) = X(c)+R$. This works because $X = G(x)$, $R = G(r)$, and $G(cx + r) = G(cx)+G(r)$, and $cx = xc$. Thus, the signature is a zero-knowledge proof of ownership of the secret key x that can be

confirmed by anyone with the public key X and the public nonce point R , proving that the owner of key x saw and signed the message m .

See research references 8 and 9 for more details. See the NaCl library for a full implementation of the low-level mathematics or code reference 3 for a high-level example.

2.5. MuSig and MuSig2 Digital Signature Schemes

MuSig is a multi-party signature construction that allows n-of-n signatures to be created securely by participants and verified using an aggregate public key using ordinary Ed25519 signature verification. MuSig2 is a slight communication complexity optimization of the original MuSig. Both versions of MuSig protect against a rogue key attack that would allow any party to impersonate the whole group, and they protect parties from the well-documented Wagner birthday attack in the case that concurrent sessions are employed.

To use MuSig, @todo finish

MuSig2 differs in the following ways:

@todo finish

See research references 11 and 12 for more details. See code reference 2 for an example of how this works in practice.

2.6. Anonymous Multi-Hop Locks

The Anonymous Multi-Hop Lock (AMHL) is a cryptographic primitive devised by Malavolta et al. in a paper titled *Anonymous Multi-Hop Locks for Blockchain Scalability and Interoperability*. The authors specified multiple ways to implement this cryptographic primitive, but for our system we will employ the construction that uses only a homomorphic one-way function constructed using the underlying mathematics of Ed25519. An AMHL is used to create a chain of pending transactions between correspondents to route a remittance, allowing for all pending transactions to clear atomically (either all clear or none do).

To construct an AMHL, the remittance originator calculates the number of hops n and selects n random clamped scalars $y = \{y_1, y_2, \dots, y_n\}$ and a point $Y_1 = G(y_1)$, then calculates a set of points by applying the one-way function of multiplication by the base point G and the group operator on the previous point: $Y = \{Y_1 = G(y_1), Y_2 = Y_1 + G(y_2), \dots, Y_n = Y_{n-1} + G(y_n)\}$. The lock setup for the first hop (remitting from originator to first correspondent in the chain) is then Y_1 . The lock setup for the i -th hop is (Y_{i-1}, Y_i, y_i) where $i \leq n$. The originator then creates a pending transaction and a signature adapter for the pending transaction using Y_1 as the public tweak value T_1 . The originator's correspondent can verify that the adapter is valid and will produce a valid signature once $t_1 = y_1$ is revealed, and it can verify that its setup is valid (i.e. that $Y_2 = Y_1 + G(y_2)$), and so that correspondent creates a pending transaction forwarding the

remittance and uses its lock setup (Y_1, Y_2, y_2) to construct an adapter signature using Y_2 as the public tweak value T_2 . This process of verification, creation of pending transaction, and construction of signature adapters cascades down the remittance route until all hops have completed the process. The originator then sends $k = \Sigma(y)$ (sum of y) to the remittee of the n -th hop, who uses it to decrypt the adapter, notifying the remitter of the n -th hop of the decrypted signature, requesting an attestation, and implicitly revealing k . The remitter of the n -th hop can then release the lock for its inbound transaction: $r_{n-1} = k - y_{n-1}$, then uses r_{n-1} to decrypt the adapter for its inbound transaction, implicitly revealing r_{n-1} to the $(n-2)$ th correspondent who calculates $r_{n-2} = y_{n-1} - y_{n-2}$. This process cascades backwards to the originator, releasing all locks and clearing all transactions in the remittance route.

See research reference 10 for more details. See code references 3 and 5 for implementations.

2.7. Signature Adapters

A signature adapter is a data structure that can be decrypted into a valid signature for a given message and can be verified in its encrypted form. This system will use Ed25519 signatures, so it will modify this signature construction to create adapters.

Recall that a key pair for Ed25519 consists of a private scalar x and a public curve point X . Recall also that a signature can be calculated for any message m by generating a nonce pair (r, R) , a challenge $c = H(R || X || m)$, and calculating a scalar $s = cx + r$. There are two methods of creating a signature adapter from this construction, and we will use the public key tweaking method: choose a random scalar t and generate a point $T = G(t)$; then the signature adapter for message m and key pair (x, X) is (R_a, T, s_a) where $s_a = cx + r_a$ and $c = H(R_a + T || X || m)$.

The adapter can be verified by checking the equality $G(s_a) = X(c) + R_a$. The adapter can be decrypted to a valid signature for m by adding R_a and T and subtracting t from s_a : $s = s_a - t$, $R = R_a + T$. The signature for message m is then (R, s) and validates as described above in section 2.2.

See research reference 13 for more details. See code references 3 and 5 for implementations.

2.8. AES and XOFCipher

The Advanced Encryption Standard (AES) is the standardized variant of the Rijndael symmetric block cipher proposed by Daemen and Rijmen. AES is designed as a substitution-permutation network using a block size of 128 bits, a key size of 128, 192, or 256 bits, and an initialization vector (IV) size of 128 bits (16 bytes). AES-256 uses a key size of 256 bits. AES is one of the most intensely scrutinized and analyzed block ciphers in existence and has withstood the test of time as the standard cipher for protecting US government TOP SECRET classified data. Many good explanations for how AES works can be found elsewhere, and this system will use it simply for symmetric encryption/decryption via the PyNaCl Sealed Box, so it is omitted here for brevity.

XOFCipher is a simple symmetric cipher created by concatenating a nonce to a secret key and using the resultant byte string as the preimage for the shake256 extensible output function (XOF). The digest length is set to the size of the plaintext to be encrypted or ciphertext to be decrypted, and the resulting bytes are simply XORed with the XOF bytes. The security of the cipher relies upon the security of the underlying XOF shake256 and the use of sufficiently large nonces and secret keys (at least 128 bits for both nonce and key) but has not been formally analyzed and evaluated. It will be included as an optional tool but will not be utilized in any protocol.

2.9. Curve25519 Elliptic Curve Diffie-Hellman Exchange

Curve25519 is a version of the Twisted-Edwards Curve compatible ("birationally equivalent") with Ed25519: an Ed25519 key pair can be converted into a Curve25519 key pair. Curve25519 was also created by Daniel J. Bernstein et al. However, while Ed25519 is an EdDSA system useful for creating signatures, Curve25519 is an elliptic curve system useful for asymmetric encryption. It accomplishes this through a technique called Elliptic Curve Diffie-Hellman Exchange (ECDHE). ECDHE works by first exchanging the public keys of the participants and deriving a shared secret (used as the seed for generating a symmetric cipher key) from one private key and the other public key. Any observer who surveils the communications will be unable to derive the shared secret, allowing any two participants to safely establish a secure communication channel over an unsecured communication channel. ECDHE relies upon the elliptic curve discrete log problem for security.

The math is simple: given two key pairs ($x, X=Gx$) and ($y, Y=Gy$), a shared secret can be calculated: $Yx = Xy$ because $X = Gx$ and $Gxy = Gyx$ (scalar-scalar and scalar-point multiplication are commutative). Without either private key (x or y), calculation of the shared secret is infeasible because $G(x)$ cannot be reversed.

2.10. Ephemeral ECDHE

Ephemeral ECDHE uses a freshly generated key pair for the sender to use when encrypting the message to be sent to a participant with a known public key. The ephemeral public key is included with the encrypted message. Ephemeral keys are discarded after use.

This system will use the ephemeral ECDHE system included in the NaCl library. The nomenclature used by the library is that the sender constructs a "Sealed Box" and the receiver "Unseals" the box to read the content.

2.11. HashCash with TapeHash

HashCash is an anti-spam mechanism created by Dr. Adam Back. It works by requiring that every message to be forwarded in a message forwarding system produce a message digest that meets a difficulty threshold when used as the preimage for a hash algorithm. The difficulty threshold is defined as a specified number of null bits at the beginning of the digest. This requires that some part of the

message, typically an integer nonce, be iteratively changed until the difficulty threshold is met. Since the output of the hash algorithm is pseudo-random, the probability that the digest includes n preceding null bits is $1/2^n$. Verifying that a message meets the difficulty threshold requires a message forwarder to calculate a single hash, which is a fast and computationally inexpensive operation. However, finding a message (by changing the nonce) that meets the difficulty threshold is a tunably slow and computationally expensive process, limiting the amount of messages that can be sent and thus preventing spam.

TapeHash is a couple of algorithms designed for use in HashCash systems that uses hash algorithms to produce a string of byte codes and an initial state (the tape) from the preimage; it interprets the byte codes to perform operations on elements from the state; it then passes the final state to sha256 and returns that digest. TapeHash1 has a configurable code size and fixed memory size, allowing it to be tuned to be computationally hard. TapeHash2 has a configurable memory size and a fixed code size, allowing it to be tuned to be memory hard. The varieties of hash algorithms used in the operations applied to the tape complicate the production of ASICs (application specific integrated circuits), and these configuration options allow the algorithms to be tuned to make ASICs irrelevant and uneconomical to produce.

This system will use TapeHash1 as the hash algorithm for HashCash to prevent network spam. Difficulty for data to be forwarded throughout the network and for messages sent to neighbors in the network will be set to target averages of 5 seconds and 1 second of computation for common computer hardware, respectively.

3. Planned Implementation

The planned implementation will be written in Python for interpreter versions 3.7+ and will make use of the PyNaCl library version 1.5+.

3.1. Contracts and Code

The contract will be embedded within the code for the system, and the terms of the contract will include the code by reference to the file hash calculated by removing the FILE_HASH definition line before hashing the content. The code/contract will take the form of a single main file that defines the most important actions and protocols of the contract and a small set of dependencies flattened into individual files, and the content hashes of those dependencies will be checked by the main file for code verification. The primary contract will handle the current ledger and base monetary unit; secondary contracts will be created to handle transactions applied to the future ledger and will reference and depend upon the primary contract.

3.2. Data Models

3.2.1. Correspondent Identity

Property Name	Data Type	Description
seed	bytes	32 byte seed used to create the key pair
private_key	nacl.signing.SigningKey	Ed25519 private key for making digital signatures
public_key	nacl.signing.VerifyKey	Ed25519 public key for verifying signatures
name	str	Name of the entity
is_natural	bool	Whether or not the entity is a human
kyc_data	dict	Dict containing KYC information, e.g. street address
kyc_commitment	bytes	32 byte Merkle root of serialized kyc_data leaves

The seed and private_key are always private and not shared. The name and kyc_data are private but will be shared with correspondents. All other properties are public and shared with the network. The public_key of a correspondent will be the identifier for that correspondent within the network.

3.2.2. Router Identity

Property Name	Data Type	Description
seed	bytes	32 byte seed used to create the key pair
private_key	nacl.signing.SigningKey	Ed25519 private key for making digital signatures
public_key	nacl.signing.VerifyKey	Ed25519 public key for verifying signatures
name	str	Name of the entity
timestamp	int	Timestamp of router creation
sponsor	bytes	The public_key of the sponsoring correspondent
sponsor_sig	bytes (optional)	Signature of the serialized router public_key, name, and timestamp by the sponsor

The seed and private_key are always private and not shared. The public_key, name, timestamp, sponsor, and sponsor_sig will be public and shared with the network. The public_key of a router will be the identifier for that router within the network. The router identifier can be the same as the sponsoring correspondent identifier.

3.2.3. Correspondence Relationship

Property Name	Data Type	Description
correspondence_id	bytes	Hash of the serialized correspondence data
contract_id	bytes	Hash of the contract governing the relationship
correspondents	list[bytes]	The public key bytes of the correspondents
multisig_key	musig2.PublicKey	The MuSig2 multisig key for the counterparties

credit_limits	dict	A dict mapping the public_key bytes for each correspondent to the integer credit limit for that correspondent in the contract unit of account
additional_terms	bytes	A serialized dict encrypted with the ECDHE shared secret of the correspondents.
commitment	bytes	The sha256 hash of the plaintext of the additional_terms.
start_timestamp	int	Unix epoch timestamp for the start of the correspondence
stop_timestamp	int (optional)	Optional Unix epoch timestamp for the end of the correspondence
signatures	list[bytes]	The signatures of the correspondence_id concatenated with the stop_timestamp by each counterparty and the multisig key

The data serialized for calculating the correspondence_id are the contract_id, correspondents, multisig_key, credit_limits, commitment, and start_timestamp. The multisig_key can be calculated by any party from the public_key bytes for each correspondent. The additional_terms field can be decrypted by either party, and the inclusion of the commitment in the signatures explicitly binds the parties to the additional terms contained therein, which will likely include collateral requirements. A correspondence can be renewed by increasing the stop_timestamp and recalculating the signatures.

3.2.4. Router Delegation

Property Name	Data Type	Description
router_id	bytes	The bytes of the router's public_key
correspondent_id	bytes	The bytes of the correspondent's public_key
start_timestamp	int	Unix epoch timestamp for the start of the delegation
stop_timestamp	int	Unix epoch timestamp for the end of the delegation
signatures	list[bytes]	The signatures of the first four elements by router and correspondent

All values are public and will be shared with the network. The maximum allowable value for stop_timestamp will be start_timestamp + 31,557,600 (1 year). Every client will run its own delegated router to enable protected storage of the primary private key.

3.2.5. Ledger

Property Name	Data Type	Description
owner_id	bytes	The bytes of the public_key of the ledger owner

aggregate_equity	int	Dynamic value calculated from the ledger entries
aggregate_future_equity	int	Dynamic value calculated from the ledger entries
aggregate_assets	int	Dynamic value calculated from the ledger entries
aggregate_future_assets	int	Dynamic value calculated from the ledger entries
aggregate_liabilities	int	Dynamic value calculated from the ledger entries
aggregate_future_liabilities	int	Dynamic value calculated from the ledger entries

The dynamic values are cached and recalculated whenever a transaction clears.

3.2.6. Account

Property Name	Data Type	Description
account_id	bytes	The hash of the serialized account details
ledger_id	bytes	The bytes of the public_key of the ledger owner
owner_id	bytes	The bytes of the public_key of the account owner
contract_id	bytes	The hash of the contract governing the account
entry_type	enum	One of "Asset", "Liability", or "Equity"
account_type	enum	One of "Starting Capital", "Equity", "Receivable", "Payable", "Common", or "Preferred"
name	str	The name of the account
balance	int	Dynamic value calculated from the ledger entries

The private values are name and balance. The rest are public values. The ledger_id, owner_id, contract_id, entry_type, and account_type are serialized and hashed to form the account_id. The balance is cached and recalculated whenever a transaction clears.

3.2.7. Ledger Entry

Property Name	Data Type	Description
ledger_id	bytes	The bytes of the public_key of the ledger owner
account_id	bytes	The hash of the serialized account details
entry_type	enum	One of "Credit" or "Debit"
amount	int	The amount of the entry

Ledger entries are not shared with the network except in the process of creating and clearing transactions.

3.2.8. Pending Transaction

Property Name	Data Type	Description
transaction_id	bytes	The hash of the serialized ledger entries, timestamp, and nonce
entries	list[LedgerEntry]	A list of ledger entries for the transaction. A typical transaction contains 4 entries referencing 2 ledgers.
timestamp	int	Unix epoch timestamp
nonce	bytes	16-byte nonce
signatures	list[bytes] (optional)	n-1 of the n signatures needed for authorization
partial_signature	musig2.PartialSignature (optional)	A partial signature for authorizing the transaction
adapter	musig2.Adapter (optional)	An adapter for completing the multiparty signature

All values will be shared between relevant correspondents. After the entries are prepared for the transaction, the receiving counterparty creates and sends either a single signature (when routing through a router delegated by a correspondent) or a partial signature (when routing through a correspondent) for the transaction id. The sending counterparty can then either provide a counter signature or complete the multiparty signature to authorize the transaction, or they may create an adapter that, once decrypted, will provide the countersignature or complete the multiparty signature.

3.2.9. Route Lock

Property Name	Data Type	Description
transaction_id	bytes	The id of the locked transaction
previous_point	bytes (optional)	The point used in the left lock
point	bytes (optional)	The current point used in the current lock
scalar	bytes	The scalar used for validating the point and calculating the unlock
previous_scalar	bytes (optional)	The scalar used for unlocking the right lock
revocation_lock	bytes	The hash lock used for revoking a remittance

All values are public and can be shared with correspondents. The previous_point will not be present for the left-most lock. The point will not be present for the right-most lock. The previous_scalar value is received when the right lock is unlocked. All route locks for a remittance route will share the same revocation_lock.

3.2.10. Cleared Transaction

Property Name	Data Type	Description
transaction_id	bytes	The hash of the serialized timestamp and ledger

		entries
entries	list[LedgerEntry]	A list of ledger entries for the transaction. A typical transaction contains 4 entries referencing 2 ledgers.
timestamp	int	Unix epoch timestamp
signatures	list[bytes] (optional)	One signature from each correspondent
multisig	musig2.Signature (optional)	Completed multiparty signature
rollup_id	bytes	The ID of the roll-up that included this txn

A cleared transaction can be authorized by either a signature from each correspondent or by a multisig. The former is useful for batch transactions with delegated routers.

3.2.11. Attestation

Property Name	Data Type	Description
digest	bytes	The hash of the data being attested
timestamp	int	Unix epoch timestamp
attester	bytes	The public_key of the attesting correspondent
signature	bytes	The 64-byte signature of the transaction_id and timestamp

For a transaction validity attestation, the digest will be equal to the transaction_id (i.e. the digest of the serialized ledger entries). All values in an attestation are public.

3.2.12. Transaction Roll-up

Property Name	Data Type	Description
rollup_id	bytes	The hash of the serialized roll-up
previous_id	bytes	The id of the previous transaction roll-up
correspondence_id	bytes	The ID of the correspondence
timestamp	int	Unix epoch timestamp
txn_root	bytes	Merkle root committing all cleared transactions for the roll-up period
balances	list[Ledger Entry]	The aggregate ledger entries for the roll-up
signature	bytes	MuSig2 signature

A transaction roll-up is a commitment to cleared transactions and the rolling forward of aggregate account balances. The previous_id, correspondence_id, timestamp, txn_root, and balances are

serialized and hashed to produce the rollup_id, and the rollup_id is signed by MuSig2 to validate the roll-up.

3.3. Suggested Yield Curve

The bundled, reference yield curve will be constructed with zero-coupon bonds and will take the following shape.

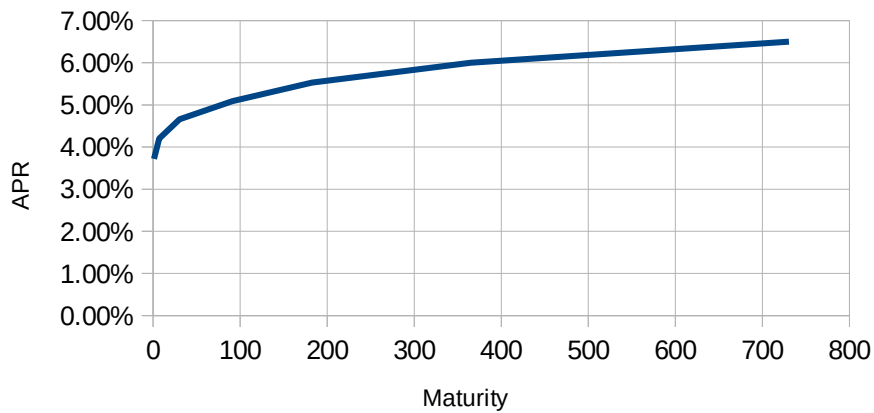
Name	Maturity in Seconds	Discount in Basis Points	APR
Current	0 (infinite)	0	0%
Day	86,400	1	3.72%
Week	604,800	8	4.20%
Month	2,629,800	38	4.66%
Quarter	7,889,400	125	5.09%
Half Year	15,778,800	273	5.53%
Year	31,557,600	600	6.00%
2 Years	63,115,200	1342	6.50%

Zero-coupon notes will be included for reference as well, and their cashflows are discounted using the above yield curve to calculate the proper discount rate for each maturity.

Name	Maturity in Seconds	Discount (BPs)	APR	Amortization Schedule
Week	604,800	7	3.72%	7 equal payments equally spaced
Month	2,629,800	33	4.03%	8 equal payments equally spaced
Quarter	7,889,400	111	4.52%	9 equal payments equally spaced
Half Year	15,778,800	234	4.73%	10 equal payments equally spaced
Year	31,557,600	521	5.21%	12 equal payments equally spaced
2 Year	63,115,200	1146	5.57%	16 equal payments equally spaced

Suggested Yield Curve

Implemented with Zero-Coupon Bonds



3.4. Protocols and Policies

3.4.1. Messaging Pub/Sub with Gossip Channels

3.4.2. Timestamps and Temporal Constraints

3.4.3. Creating Identities

3.4.4. Establishing Correspondences

3.4.5. Monetizing Natural Person Correspondent Identities

3.4.6. Remitting Value Between Correspondents

3.4.7. Funding Non-Natural Person Correspondent Identities

3.4.8. Delegating Routing Permission

3.4.9. Spanning Tree Construction

3.4.10. Message and Remittance Routing

3.4.11. Remittance Routing by Delegates

3.4.12. Clearing

3.4.13. Ledger Aggregate Querying

3.4.14. Attestations

3.4.15. Transaction Roll-ups

3.5. Cryptography

The cryptographic primitives used for the below constructions will be provided by the hashlib and PyNaCl libraries.

3.5.1. SHA256, SHA512, and Shake256

Sha256, sha512, and shake256 can be imported directly from the hashlib library. The PyNaCl library uses sha512 for some of its digital signature operations.

3.5.2. Merkle Trees and Merkle Mountain Ranges

Merkle structures will be implemented using sha256 and a simple class. A class method will automate the creation of tree structures, and custom mountain range structures will be possible by joining branches together arbitrarily. Serialization and deserialization will use the json library. See code reference 1 for the code intended to be used for this purpose.

3.5.3. Directed Acyclic Graphs

3.5.4. Ed25519 Digital Signature Scheme

3.5.5. MuSig/MuSig2 Digital Signature Scheme

3.5.6. Anonymous Multi-Hop Locks

3.5.7. Curve25519 Elliptic Curve Diffie Hellman Exchange

3.5.8. Ephemeral ECDHE

3.5.9. HashCash with TapeHash

3.6. Optimizations

4. Security Considerations

There are multiple security threats that must be addressed in any distributed system. In this section, several common threats will be discussed and the planned implementation evaluated for resilience and robustness.

4.1. Sybil Attacks

One of the primary issues in peer-to-peer (p2p) distributed systems is the potential for being the target of a Sybil attack, where one entity pretends to be many hundreds or thousands of participants either by creating fictitious identities or by running many nodes. Systems like Bitcoin that use proof-of-work consensus are strengthened by such attempts because the increase in competition for the right to mint blocks increases the thermodynamic hardness of the block minting process. However, the only consensus used in this system is created between nodes that have explicit trust relationships with each other (implicit for exactly one scenario where a node drops from the network and its correspondents mark it inactive). Due to the nature of friend-to-friend (f2f) network overlays, the effect of Sybil attacks is muted. I will demonstrate this by reviewing several processes in the process of a Sybil actor.

4.1.1. Sybil Attack in Spaning Tree Construction

Given a set of real users and a set of Sybil users, the only way for a Sybil user to affect the spanning tree construction is to participate in it honestly. If a Sybil user recalculates its key pair until it elects itself root or takes an intermediate position in the spanning tree but refuses to allocate addresses to real users, the real user spanning trees will not be affected because root elections will only be forwarded by nodes that have received an address allocation from that spanning tree. In the case of a Sybil user taking the root position, it will form a tree only with other Sybil users if it refuses to include real users, so the real users will be unaffected. In the case of a Sybil user taking an intermediate position in the spanning tree, real users will simply get addresses from other real users if the Sybil user refuses to allocate child addresses.

4.1.2. Sybil Attacks in Remittance Routing

More generally, because the f2f overlay is constructed of trust relationships, the only real users affected by a Sybil attack will be those that blindly trust Sybil users. The requirement for collecting KYC data by participants limits the scope of damage that can be caused by Sybil users. For remittance routing in particular, a potential concern is that Sybil users could inflate the number of hops needed to route a remittance and thus increase latency and costs. This can be alleviated by displaying possible remittance routes before a payment is made, allowing a legitimate user to choose the route with the lowest fee and/or latency.

4.2. Spam/Denial of Service

Network spam can be alleviated by requiring some small proof-of-work/hashcash to be included in protocol messages. The difficulty threshold can be automatically adjusted as a matter of node policy: in the event of increased network traffic, an intermediate node can increase its difficulty threshold and refuse to forward messages that do not meet that threshold. This will introduce a computational bottleneck that preserves network throughput from spammers, and it will increase that bottleneck closest to the source of the spam, allowing the network to function normally in areas not affected by spammers. A more complex system for detecting spam and adjusting per-address difficulty threshold can be implemented as a stretch goal, though this will entail higher hardware requirements for nodes.

4.3. Crash Faults

As the only state that must be maintained in secrecy are private keys, recovery from a node crash can be accomplished by synchronizing transaction history with correspondents. The use of Bloom filters can substantially increase the efficiency of such synchronization processes, and correctness can be determined by comparing root commitments calculated at each node by sorting and hashing the transaction IDs.

4.4. Byzantine Faults

@todo

4.5. Hot/Cold Wallets

Since the identity on the network is controlled by an ed25519 private key, keeping this private key loaded on potentially insecure devices (e.g. phones or point-of-sales machines) will compromise the security of the identity if the device becomes compromised, allowing an adversary the ability to spend all available liquidity, engage in Sybil/Byzantine behaviors, attempt to defraud correspondents, or cause other forms of harm to the participant. An easy and more secure alternative exists: use fictitious correspondents with unidirectional credit limits to receive and spend deposits held with the primary identity to form a "hot wallet". These transactions can be routed through a primary or secondary router without risking the identity private key or any unwanted spending. The private key for the identity can be stored in a hardware wallet to separate it from any online devices for greater security. Funds could then be sent from the hot wallet to the cold wallet to protect them or from the cold wallet to the hot wallet to spend. A hardware wallet for the identity private key, a primary router delegation, and a deposit-mode "hot wallet" will be the recommended setup.

5. Development Roadmap

- Adapter signature proof-of-concept: complete
- Anonymous multi-hop lock proof-of-concept: complete
- Content hashing proof-of-concept: complete
- Gossip protocol proof-of-concept: complete
- Double-entry accounting logic: complete
- MuSig: complete
- MuSig adapters: pending
- MuSig2 (with adapters): pending
- Anonymous multi-hop locks: complete
- SpeedyMurmurs: pending
- Authorization script system: complete
- Script-authorized double-entry: pending
- Contracts proof-of-concept (Python): complete
- Contracts proof-of-concept (Lua): pending
- Primary contract: blocked
- Secondary contracts - equity: blocked
- Secondary contracts - bonds: blocked
- Secondary contracts - notes: blocked
- Atomic swaps: blocked
- Reference implementation - backend: blocked
- Reference implementation - console app: blocked

- Reference implementation - graphical app: blocked

6. References

Information and inspiration for this system were found in the research sources listed below.

1. *Anonymous Addresses for Efficient and Resilient Routing in F2F Overlays* by Steanie Roos, Marin Beck, and Thorsten Strufe, IEEE INFOCOM 2018
2. *VOUTE: Virtual Overlays Using Tree Embeddings* by Steanie Roos, Martin Beck, and Thorsten Strufe, arXiv:1601.06119v1 [cs.CR] 22 Jan 2016
3. *Settling Payments Fast and Private: Efficient Decentralized Routing for Bath-Based Transactions (SpeedyMurmurs)* by Stefanie Roos, Pedro Moreno-Sanchez, Anikey Kate, and Ian Goldberg, arXiv:1709.05748v2 [cs.CR] 13 Dec 2017
4. *Attack-resistant Spanning Tree Construction in Route-Restricted Overlay Networks* by Martin Byrenheid, Stefanie Roos, and Thorsten Strufe, arXiv:1901.02729v5 [cs.CR] 9 Aug 2019
5. *An Algorithm for Distributed Computation of a Spanning Tree in an Extended LAN* by Radia Perlman, 1985 ACM 0-89791-164-4/85/0009/0044800.75
6. *R/Kademlia: Recursive and Topology-aware Overlay Routing* by Bernhard Heep, 2010 Australasian Telecommunication Networks and Applications Conference
7. *Exploiting Centrality: Attacks in Payment Channel Networks with Local Routing* by Ben Weintraub, Cristina Nita-Rotaru, and Stefanie Roos, arXiv:2007.09047v1 [cs.CR] 17 Jul 2020
8. *High-speed high-security signatures* by Daniel J. Bernstein, Niels Duij, Tanja Lange, Peter Schwabe, and Bo-Yin Yang, 2011.09.26
9. *Edwards-Curve Digital Signature Algorithm (EdDSA)* by S. Josefsson et al., IETF RFC 8032 January 2017
10. *Anonymous Multi-Hop Locks for Blockchain Scalability and Interoperability* by Giulio Malavolta, Pedro Moren-Sanchez, Calra Schneidewind, Anikey Kate, and Matteo Maffei, NDSS Symposium 2019
11. *Simple Schnorr Multi-Signatures with Applications to Bitcoin (MuSig)* by Gregory Maxwell, Andrew Poelstra, Yannick Seurin, and Pieter Wuille, 2018 May 20
12. *MuSig2: Simple Two-Round Schnorr Multi-Signatures* by Jonas Nick, Tim Ruffing, and Yannick Seurin, CRYPTO 2021 revised 2021-07-06
13. *Adaptor signature - Schnorr signature and ECDSA* by Ichiro Kuwahara, Medium.com October 8, 2020.

Reference code can be found in the sources listed here:

1. Merkle tree implementation: <https://pypi.org/project/merkleasy>
2. MuSig implementation: <https://github.com/k98kurz/musig>
3. Ed25519 + signature adapter + AMHL proof-of-concept: <https://pastebin.com/HKUhtY1E>
4. Gossip implementation including TapeHash algorithms: <https://github.com/k98kurz/gossip>

5. Tapescript bytecode VM system: <https://pypi.org/project/tapescript>

The tapescript package includes AMHLs, signature adapters, point time locks, and an e2e test combining and verifying the three.

7. Glossary

Clearing - the process of validating and recording transactions

Entry - a piece of data specifying an account on a ledger, an amount, and the type (credit or debit)

Transaction - a collection of ledger entries and relevant digital signatures/authorization scripts

8. Additional Examples

9. License

ISC License adapted for a software specification. All software developed by the author in pursuit of developing and implementing this specification is also released under ISC or an equivalent license.

Copyright (c) 2024 Jonathan Voss

Permission to use, copy, modify, and/or distribute this software specification for any purpose with or without fee is hereby granted, provided that the above copyright notice and this permission notice appear in all copies.

THE SOFTWARE SPECIFICATION IS PROVIDED "AS IS" AND THE AUTHOR DISCLAIMS ALL WARRANTIES WITH REGARD TO THIS SOFTWARE SPECIFICATION INCLUDING ALL IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS. IN NO EVENT SHALL THE AUTHOR BE LIABLE FOR ANY SPECIAL, DIRECT, INDIRECT, OR CONSEQUENTIAL DAMAGES OR ANY DAMAGES WHATSOEVER RESULTING FROM LOSS OF USE, DATA OR PROFITS, WHETHER IN AN ACTION OF CONTRACT, NEGLIGENCE OR OTHER TORTIOUS ACTION, ARISING OUT OF OR IN CONNECTION WITH THE USE OR PERFORMANCE OF THIS SOFTWARE SPECIFICATION.

10. Final Notes

The author welcomes any contributions to this FOSS project, including suggestions, corrections, software code, or material assistance offered. The goal of this project is to benefit all of humanity with a freely provided, secure, and truly decentralized monetary system to complement existing systems and to enable communities to recover from collapses of fiat currencies which history indicates are inevitable.

The author can be reached on the Pycelium discord server: <https://discord.gg/b2QFEJDX69>